Санкт-Петербургский государственный университет

ЧИЖОВ Антон Игоревич

Выпускная квалификационная работа

Реализация поддержки приближенных зависимостей включения и зависимостей включения подобия в рамках профилировщика Desbordante

Уровень образования: магистратура

Направление *02.04.03 «Математическое обеспечение и администрирование информационных систем»*

Основная образовательная программа *ВМ.5665.2023 «Математическое обеспечение и администрирование информационных систем»*

Научный руководитель:
к. ф.-м. н., старший научный сотрудник кафедры информационно-аналитических систем
Е. Г. Михайлова

Консультант:
ассистент кафедры информационно-аналитических систем Чернышев Г. А.

Рецензент:
программист-разработчик, ООО «В КОНТАКТЕ» Слесарев А. Г.

Санкт-Петербург
2026

Saint Petersburg State University

Anton Chizhov

Master's Thesis

Supporting approximate and similarity inclusion dependencies in Desbordante data profiler

Education level: master

Speciality *02.04.03 "Software and Administration of Information Systems"*

Programme *BM.5665.2023 "Software and Administration of Information Systems"*

Scientific supervisor:
senior researcher, C.Sc. E. G. Mikhailova

Consultant:
assistant G. A. Chernishev

Reviewer:
programmer-developer at "V KONTAKTE" A. G. Slesarev

Saint Petersburg
2026

Оглавление

Введение **4**

1. Постановка задачи **6**

2. Обзор предметной области **7**

2.1. Основные определения 7

2.2. Обзор существующих подходов к обработке неточностей в данных 7

2.3. Сравнение IND, AIND и SIND 10

2.4. Существующие инструменты 12

2.5. Платформа Desbordante 12

3. Обзор алгоритмов **14**

3.1. Алгоритм Spider 15

3.2. Алгоритм Mind 17

3.3. Алгоритм Sawfish 20

3.4. Выводы 24

4. Реализация **25**

4.1. Алгоритм Spider 26

4.2. Алгоритм Mind 30

4.3. Алгоритм Sawfish 37

4.4. Итоги реализации 39

5. Эксперименты **40**

5.1. Методология 40

5.2. Алгоритм Spider 41

5.3. Алгоритм Mind 44

5.4. Алгоритм Sawfish 47

Заключение **51**

Список литературы **52**

Введение

Профилирование данных [1] является комплексным аналитическим процессом, цель которого заключается в выделении ключевых метаданных. Особый интерес представляет профилирование, направленное на обнаружение зависимостей в данных. Наиболее известный пример — функциональные зависимости, которые являются ограничениями целостности в базах данных [3]. Такие зависимости, например, позволяют проверять, приведено ли отношение к третьей нормальной форме [3].

Хотя ограничения целостности, такие как функциональные зависимости, успешно описывают семантику данных, некоторые данные в базе данных могут не соответствовать этим ограничениям. Это может происходить, например, при интеграции данных из различных независимых источников. Для таких случаев могут использоваться зависимости включения [7, 8] (Inclusion Dependency, IND). В отличие от традиционных функциональных зависимостей, зависимости включения определяются между двумя отношениями и позволяют проверять согласованность данных в них.

Неформально, между двумя наборами колонок из двух (возможно, совпадающих) отношений существует зависимость включения, если для каждого кортежа из первого набора атрибутов в первом отношении существует такой же кортеж из второго набора атрибутов во втором отношении.

Зависимости включения могут [2] использоваться для обнаружения несогласованности данных и выявления нарушений целостности. Однако круг задач, которые на практике могут быть решены с помощью точных зависимостей включения, сильно ограничен. Например, такие зависимости не могут использоваться для обнаружения отсутствующих ограничений и проблем с целостностью данных. Для решения подобных задач могут использоваться расширения зависимостей включения, которые определяются подобно расширениям функциональных зависимостей [9]. Поддержка таких расширений — важная составляющая функциональности современного профилировщика данных.

Desbordante [6] представляет собой научно-ориентированный профилировщик данных с открытым исходным кодом[1]. Создание Desbordante было мотивировано недостатками существующей платформы Metanome [5]. В Desbordante в качестве основного языка используется C++, а в Metanome — Java, из-за чего Metanome имеет не самую оптимальную производительность. Desbordante включает в себя алгоритмы для поиска различных примитивов и предоставляет соответствующие интерфейсы для пользователей.

В настоящее время Desbordante активно развивается как инструмент анализа данных и требует расширения набора поддерживаемых примитивов, в том числе расширений зависимостей включения, необходимых для обработки данных, содержащих ошибки и опечатки. Реализация поддержки таких расширений была поставлена в качестве задачи перед автором данной работы.

[1] https://github.com/Desbordante/desbordante-core

1. Постановка задачи

Целью данной работы является расширение поддержки зависимостей включения для работы с данными, содержащими ошибки. Для её выполнения были поставлены следующие задачи.

- Выполнить обзор предметной области и выбрать подходы к обработке неточностей в данных.
- Выбрать алгоритмы для реализации.
- Реализовать выбранные алгоритмы в рамках платформы Desbordante.
- Провести экспериментальное исследование алгоритмов.

2. Обзор предметной области

В данном разделе вводятся базовые определения и рассматриваются подходы к обработке неточностей в данных с помощью зависимостей включения.

2.1. Основные определения

Определение 1. ***Зависимость включения (IND)*** *определяется следующим образом:*

$$IND : R_1(X) \subseteq R_2(Y),$$

где X и Y это наборы атрибутов из отношений R_1 и R_2 соответственно. Это означает, что для каждого кортежа с атрибутами X в отношении R_1 существует такой же кортеж с атрибутами Y в отношении R_2.

Другими словами, $\forall t_1 \in \pi_X(R_1)\ \exists t_2 \in \pi_Y(R_2) : t_1 = t_2$.

Если множества X и Y содержат по одному атрибуту, то зависимость называется ***унарной****; если больше одного — n****-арной****.*

Определение 2. ***Поиском зависимостей включения*** *называется нахождение множества всех зависимостей включения, удерживающихся на заданном наборе отношений.*

Определение 3. ***Унарными*** *называются алгоритмы, предназначенные для поиска унарных зависимостей включения, а n****-арными*** *— для поиска n-арных зависимостей.*

2.2. Обзор существующих подходов к обработке неточностей в данных

Подход на основе минимальных изменений [2] (minimum change integrity maintenance) предполагает, что множество ограничений, которым должны удовлетворять данные, заранее известно. В рамках этого

подхода база данных модифицируется с помощью минимально возможных изменений (вставки или удаления кортежей), таким образом, чтобы восстановить соблюдение ограничений, например, зависимостей включения. Однако необходимость заранее задавать ограничения такие как зависимости включения сужает область применения метода и делает его менее пригодным в случаях, когда структура данных заранее неизвестна или может изменяться.

В случаях, когда ограничения заранее неизвестны или данные содержат ошибки и неполноту, логичным решением становится автоматический поиск зависимостей включения непосредственно в самих данных. Алгоритмы обнаружения зависимостей позволяют выявить потенциальные ограничения, даже при наличии неточностей, что расширяет возможности последующей их обработки и исправления. Таким образом, использование таких алгоритмов помогает адаптировать модель ограничений к реальному состоянию данных и повысить качество их очистки и восстановления.

Подобно статистическим расширениям функциональных зависимостей существуют расширения зависимостей включения, например приближённые зависимости включения (Approximate IND, AIND). С помощью таких зависимостей можно [12] обнаружить больше нарушений и ошибок в данных, особенно при наличии неточностей

Определение 4. ***Приближённая зависимость включения*** *определяется следующим образом:*

$$AIND : R_i(X) \subseteq_\varepsilon R_j(Y),$$

где пороговое значение ε задаёт максимальное значение ошибки для функции g_3', т.е. $g_3'(R_i(X) \subseteq R_j(Y)) \leq \varepsilon$.

Пусть d это база данных над схемой базы данных R и r_i, $r_j \in d$ для соответствующих отношений R_i, $R_j \in \boldsymbol{R}$. Тогда функция g_3' вычисляет соотношение минимального количества кортежей, которые необходимо удалить из r, чтобы $R_i(X) \subseteq R_j(Y)$ выполнялась над r.

$$g_3'(R_i(X) \subseteq R_j(Y)) =$$
$$= 1 - \frac{max\{|\pi_X(r')| : r' \subseteq r_i \ u \ (d - \{r_i\}) \cup \{r'\} \models R_i(X) \subseteq R_j(Y)\}}{|\pi_X(r_i)|}$$

r' это подмножество кортежей в r_i, которые не нарушают зависимость.

По данным автора данной работы, существует только один алгоритм для решения задачи поиска приближённых зависимостей включения. В статье [13] предложен алгоритм MIND для поиска унарных и n-арных зависимостей включения, а также способ расширения алгоритма для поиска приближённых зависимостей. Приближённые зависимости включения могут использоваться, например, для выявления отсутствующих ограничений и проблем с целостностью данных.

Зависимости включения являются одним из ключевых типов зависимостей, используемых в практике проектирования и сопровождения баз данных. В статье [12] предлагается метод для обнаружения приближённых зависимостей включения с учетом возможных несоответствий в данных. Полученные зависимости могут использоваться, например, для автоматической настройки логической структуры баз данных, выявления отсутствующих ограничений и упрощения задач администрирования.

В ряде случаев “глобальные” зависимости включения оказываются излишне жёсткими. Это требует уточнения с учётом контекста. Одним из решений являются условные зависимости включения (Conditional INDs, CINDs) — расширение традиционных IND, применяемое только к подмножествам строк, удовлетворяющих определённым условиям. Подобные зависимости полезны при очистке данных, контекстном сопоставлении схем и восстановлении информации [4].

Однако использование CIND сопряжено с теми же ограничениями, что и подход, основанный на минимальных изменениях: для их применения в задачах очистки и восстановления данных предполагается, что соответствующая зависимость должна удерживаться в данных. В связи с этим в рамках рассматриваемой в данной работе задачи данный тип

зависимостей не представляет практического интереса.

Последним на сегодняшний день известным расширением зависимостей включения являются зависимости включения подобия (Similarity IND, SIND) [11], основанные на использовании метрик сходства, таких как расстоянии Левенштейна и коэффициента Жаккара.

Определение 5. ***Зависимость включения подобия (SIND [11])*** *определяется следующим образом:*

$$SIND : R_1[X] \subsetneq_\sigma R_2[Y],$$

где:

1. *X и Y являются наборами атрибутов отношений R_1 и R_2, соответственно.*

2. $\forall t_1 \in \pi_X(R_1)\ \exists t_2 \in \pi_Y(R_2) : t_1 \approx_\sigma t_2$

   Другими словами, для каждого зависимого значения существует подобное ссылочное значение с учётом меры и порога ошибки.

Таким образом, среди рассмотренных расширений зависимостей включения наибольший интерес для задач анализа и восстановления данных, содержащих ошибки, представляют приближённые зависимости включения (Approximate IND, AIND) и зависимости включения подобия (Similarity IND, SIND). Обе разновидности позволяют учитывать неточности в данных и ориентированы на устойчивое обнаружение закономерностей в условиях реальных, шумных или неполных наборов данных.

2.3. Сравнение IND, AIND и SIND

Для сравнения различных зависимостей включений рассмотрим следующие таблицы. В таблице 1 определяется соответствие между курсами и преподавателями, которые ведут эти курсы, а в таблице 2 содержится информация о курсах, которые выбрали студенты. При этом

name	instructor
Machine Learning	Dr. Smith
Data Structures	Dr. Johnson
Algorithms	Dr. Brown
Databases	Dr. Lee

Таблица 1: courses

student_id	course_name
1024	Machine Learning
1031	Machine Learnigg
1045	Data Structures
1077	Algoritms
1102	Databases
1140	Data Structurms

Таблица 2: enrollments

часть студентов допустила опечатки в названиях курсов (ячейки с ошибочными значениями выделены красным цветом).

Предположим, что мы хотим исследовать следующую зависимость:

$$enrollments[course_name] \subseteq courses[name]$$

IND. Среди зависимых значений встречаются опечатки, из-за чего зависимость не выполняется. В таких случаях использование точных зависимостей включения не помогает анализировать данные.

AIND. В случае приближённых зависимостей необходимо подсчитать ошибку, с которой зависимость удерживается. Множество уникальных зависимых значений равняется шести, а количество неправильных уникальных значений равняется трём. Следовательно, ошибка составляет $\frac{3}{6} = 0.5$. Соответственно для того, чтобы обнаружить данную зависимость, необходимо использовать низкий порог ошибки. Тем не менее, использование такого низкого порога ошибки на практике приводит к большому количеству лишних зависимостей, которые не несут дополнительной информации и, скорее всего, являются случайными совпадениями. Проблема заключается в том, что приближённые зависимости включения не учитывают особенности ошибок и не дают никакой дополнительной информации о них.

SIND. Для зависимости включения подобия будем использовать расстояние Левенштейна. Можно заметить, что в зависимой части в каждом ошибочном значении содержится максимум одна опечатка. Соответственно данная зависимость будет найдена с минимальным порогом ошибки, т. е. при $\sigma = 1$. Для работы с такими данными целесообраз-

но применять зависимости включения подобия с различными мерами схожести, адаптированными к природе данных.

2.4. Существующие инструменты

Metanome[2] — это открытый инструмент на Java для профилирования данных, поддерживающий различные типы зависимостей, включая зависимости включения. Он объединяет несколько алгоритмов для поиска этих зависимостей, позволяя запускать их на одном наборе данных через интерфейс для разработки и интеграции новых алгоритмов, что делает его полезным для пользователей и разработчиков.

В кодовой базе Metanome содержатся Java-реализации всех поддерживаемых алгоритмов, включая поиск зависимостей включения и зависимостей включения подобия. В частности, доступен код алгоритма Sawfish, что позволяет использовать Metanome в качестве базы для реализации алгоритма. В то же время алгоритм Mind реализован только для поиска точных зависимостей и предназначен для работы с СУБД, без поддержки работы с `csv`-таблицами.

2.5. Платформа Desbordante

Desbordante — это наукоёмкий, высокопроизводительный профилировщик данных с открытым исходным кодом, написанный на языке C++. Основная цель платформы — объединить различные примитивы для расширения возможностей анализа данных и решения более широкого спектра задач.

Ключевым преимуществом Desbordante является высокая производительность, достигаемая за счёт использования возможностей языка C++. Это выгодно отличает платформу от Metanome, полностью написанной на Java, где производительность в ресурсоёмких задачах зачастую ограничена.

Для удобства пользователей Desbordante предоставляет несколь-

[2] https://github.com/HPI-Information-Systems/Metanome

ко интерфейсов: консольный, веб-интерфейс[3] и Python-обёртку. Ядро платформы реализует алгоритмы поиска и верификации зависимостей, а также предоставляет API для их конфигурации и запуска.

Таким образом, платформа Desbordante ориентирована как на исследователей и аналитиков, так и на разработчиков, предлагая сочетание высокой производительности и удобства использования.

[3] https://desbordante.unidata-platform.ru/ (дата обращения: 31 мая 2025 г.).

3. Обзор алгоритмов

По результатам обзора существующих подходов в подразделе 2.2 было принято решение реализовать поддержку алгоритмов для работы с приближенными зависимостями включения и зависимостями включения подобия.

Для поиска SIND на данный момент известен только один алгоритм — Sawfish. Его реализация находится в открытом доступе, что делает его единственным логичным кандидатом для интеграции.

Аналогично для поиска приближённых n-арных зависимостей включения логичным выбором является Mind. Этот алгоритм реализует поуровневый обход решётки кандидатов снизу вверх с применением эффективного отсечения неподходящих вариантов. Он использует информацию о найденных ранее унарных зависимостях для сокращения пространства поиска и тесно интегрируется с СУБД, которая отвечает за генерацию и проверку кандидатов.

Однако в контексте работы вне СУБД возникает сложность: для эффективного поиска унарных зависимостей необходима предварительная классификация столбцов по типам данных. Внутри СУБД эта информация, как правило, уже известна, что упрощает группировку кандидатов и ускоряет их проверку. Вне СУБД же определение типов требует дополнительной обработки, а смешанные типы данных в столбцах сильно усложняют проверку кандидатов.

В работе [10] представлен систематический обзор алгоритмов поиска IND, которые делятся на унарные и n-арные. Унарные алгоритмы обычно используют статистические характеристики столбцов для отсечения большого количества кандидатов и снижения числа затратных проверок. Для повышения эффективности применяются различные структуры данных — от инвертированных индексов до алгоритмов дисковой сортировки (`disk-based sort-merge-join`).

В результате анализа существующих алгоритмов поиска IND был обнаружен алгоритм Spider, который по своей структуре близок к Mind (для поиска унарных зависимостей) и при этом достаточно прост для

адаптации вне СУБД. Важно отметить, что Spider реализован в инструменте Metanome и распространяется с открытым исходным кодом, что упрощает его использование и адаптацию. Главное преимущество Spider — возможность его обобщения для поиска приближённых зависимостей включения, что делает этот алгоритм подходящей основой для собственной реализации.

Исходя из этого, в рамках данной работы было решено сначала реализовать модифицированную версию алгоритма Spider, а затем приступить к интеграции Mind — алгоритма для поиска n-арных приближённых зависимостей включения.

3.1. Алгоритм Spider

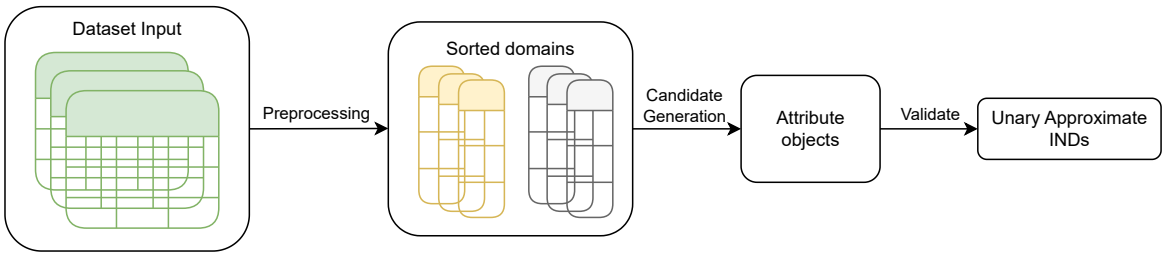

Рис. 1: Алгоритм Spider

На Рис. 1 представлены этапы работы алгоритма Spider, включающие обработку входных данных, генерацию кандидатов и их проверку.

1. Обработка входных данных.

На этой стадии Spider последовательно обрабатывает колонки входных таблиц. Обработка колонки заключается в поиске её отсортированного домена. По итогам работы алгоритма для каждой колонки создаётся отдельный колоночный файл на диске.

При этом на данном этапе алгоритм может выполнять запись промежуточных результатов на диск при достижении ограничений по памяти с последующим их соединением (схожий подход используется операторами внешней сортировки в СУБД). Далее промежуточные результаты (отсортированный домен для части таблицы) будут называться партициями.

2. Генерация кандидатов.

Для каждой колонки создаётся объект атрибута. Он содержит:

- итератор по отсортированному домену значений колонки (т.е. итерация по строкам колоночного файла);
- два списка атрибутов — список ссылочных и список зависимых атрибутов.

Таким образом, каждый объект атрибута хранит информацию о том, какие атрибуты могут находится в левой и правой частях предполагаемой зависимости. Два списка необходимы для того, чтобы прекращать обработку объекта атрибута на самом раннем этапе. Объект атрибута считается обработанным, если либо все значения колоночного файла были прочитаны, либо оба списка стали пустыми.

На данной стадии, Spider инициализирует все объекты атрибутов с итераторами, указывающими на первые значения соответствующих колоночных файлов. Оба списка кандидатов заполняются всеми атрибутами за исключением текущего. Более продвинутая версия алгоритма может уточнять данные списки кандидатов, используя информацию о типах колонок (например, нет смысла проверять зависимость между числовым типом с плавающей запятой и целочисленным типом). Базовая версия работает со всеми значениями как со строками.

3. Проверка кандидатов

Объекты атрибутов хранятся в двоичной куче (min-heap), где значением вершины является текущее значение, на которое указывает итератор. Spider выполняет следующий алгоритм до тех пор пока куча не станет пустой.

- Из вершины кучи достаётся набор объектов атрибутов, имеющих одинаковое значение.
- Для каждого объекта атрибута, принадлежащего набору, его список зависимых атрибутов пересекается с набором. Другими словами, алгоритм удаляет атрибуты, которых нет в наборе из списка зависимых атрибутов. Соответствующие ссылочные атрибуты

удаляются аналогичным образом (если для объекта атрибута A из списка кандидатов удаляется зависимый кандидат B, тогда для B удаляется ссылочный кандидат A).

- Для каждого необработанного объекта атрибута из набора, алгоритм продвигает итератор и вставляет объект атрибута в кучу.

3.2. Алгоритм Mind

Алгоритм Mind [13] является n-арным алгоритмом для поиска AIND. Как упоминалось ранее, для поиска унарных зависимостей используется модифицированный алгоритм Spider (модификации рассматриваются в подразделе 4.1).

```
1  # Input: d a database, and I_1 the set of unary INDs.
2  # Output: all INDs satisfied by d.
3  C_2 := gen_next(I_1);
4  i := 2;
5  while C_i != do
6    forall I ∈ C_i do
7        if d ⊨_ε I then
8          I_i := I_i ∪ {I};
9    C_{i+1} := gen_next(I_i);
10   i := i + 1;
11 end while
12 return ⋃_{j<i} I_j
```

Листинг 1: Алгоритм Mind.

Процесс проверки зависимостей осуществляется по уровням. Сначала формируется набор кандидатов, основываясь на зависимостях, найденных на предыдущем уровне. Затем происходит последовательная проверка кандидатов.

На Листинге 2 представлен алгоритм генерации кандидатов.

```
1  # Input: 𝓘_i, INDs of size i.
2  # Output: C_{i+1}, candidates of size i+1.
3  INSERT INTO C_{i+1}
4  SELECT p.lhs.rel[p.lhs[1], p.lhs[2], ..., p.lhs[i], q.lhs[i]]
5         ⊆ p.rhs.rel[p.rhs[1], p.rhs[2], ..., p.rhs[i], q.rhs[i]]
6  FROM 𝓘_i p, 𝓘_i q
7  WHERE p.lhs.rel = q.lhs.rel AND p.rhs.rel = q.rhs.rel
8      AND p.lhs[1] = q.lhs[1] AND p.rhs[1] = q.rhs[1]
9      AND ...
10     AND p.lhs[i−1] = q.lhs[i−1] AND p.rhs[i−1] = q.rhs[i−1]
11     AND p.lhs[i] < q.lhs[i] AND p.rhs[i] <> q.rhs[i]
12 for all I ∈ C_{i+1} do
13   for all J ≺ I and J of size i do
14     if J ∉ 𝓘_i then
15       C_{i+1} := C_{i+1} \ {I}
16     end if
17   end for
18 end for
```

Листинг 2: Алгоритм генерации кандидатов.

Для генерации кандидатов $i+1$ уровня перебираются всевозможные комбинации зависимостей i уровня. Зависимости должны отличаться только последним атрибутом. На завершающем этапе для каждого кандидата проверяется, что все его подзависимости размера i выполняются. На Листинге 3 представлен упрощённый, более высокоуровневый, алгоритм генерации кандидатов.

```
1  # Input: 𝓘_i, INDs of size i.
2  # Output: C_{i+1}, candidates of size i+1.
3  forall (p, q) ∈ 𝓘_i × 𝓘_i:
4      if not p.startswith(q):
5          continue
6
```

```
7       if not (p.lhs.last_column() < q.lhs.last_column() and
8               p.rhs.last_column() ≠ q.rhs.last_column()):
9               continue
10
11      candidate := create_candidate(p, q)
12      if can_prune_candidate(candidate, prev_raw_inds):
13          continue
14
15      C_{i+1} := C_{i+1} ∪ {candidate}
16  end for
```

Листинг 3: Упрощенный алгоритм генерации кандидатов.

Последняя часть алгоритма заключается в проверке кандидата. В оригинальной статье [13] приведён запрос (листинг 4).

```
1  SELECT * FROM r
2  WHERE NOT EXISTS (
3      SELECT * FROM s
4      WHERE (r.A_1, ..., r.A_n) = (r.B_1, ..., r.B_n)
5  );
```

Листинг 4: Проверка точного кандидата.

Фактически, для проверки точной зависимости необходимо выполнить проекцию обоих отношений: в одном случае — на атрибуты, которые используются в левой части зависимости, в другом — на атрибуты в правой части. Затем необходимо проверить, что все кортежи из левой проекции также присутствуют в правой.

Для поиска приближенных зависимостей предлагается использовать следующие запросы, представленные в Листинге 5. Для проверки корректности кандидата достаточно поделить количество уникальных кортежей ($disqualifying_rows$), которые не содержатся в правом отношении, на мощность левого множества ($lhs_cardinality$). Неформально значение ошибки для зависимости включения можно определить как число строк (независимо от количества их повторений), которые необ-

ходимо удалить из левого отношения, чтобы получить набор данных, на котором зависимость выполняется.

```
1 # count error rows
2 SELECT COUNT(DISTINCT r.A_1, ..., r.A_n) AS disqualifying_rows
3 FROM r
4 WHERE NOT EXISTS (
5     SELECT * FROM s
6     WHERE (r.A_1, ..., r.A_n) = (r.B_1, ..., r.B_n)
7 );
8 # calculate lhs cardinality
9 SELECT COUNT(DISTINCT r.A_1, ..., r.A_n) FROM r;
```

Листинг 5: Проверка приближенного кандидата.

3.3. Алгоритм Sawfish

Алгоритм Sawfish (**S**imilarity **AW**are **F**inder of **I**nclusion dependencies via a **S**egmented **H**ashindex) [11] является единственным известным на данный момент алгоритмом для поиска унарных зависимостей включения подобия.

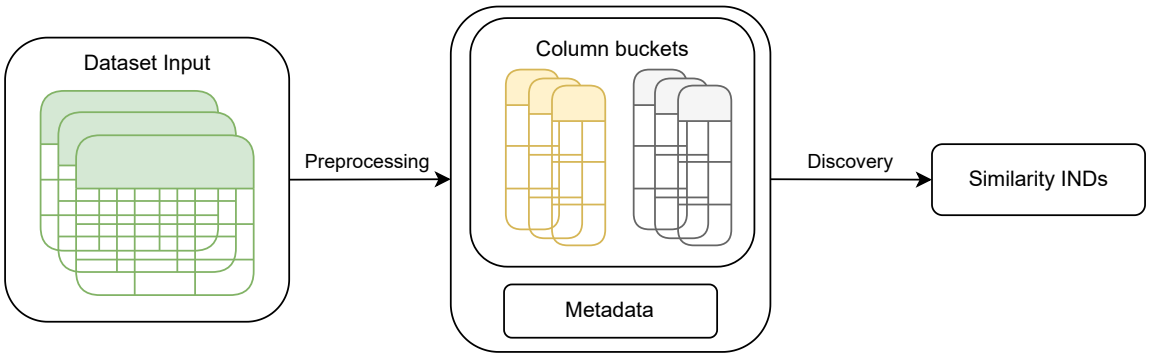

Рис. 2: Алгоритм Sawfish

На Рис. 2 показаны стадии работы алгоритма Sawfish. На первом этапе входные данные преобразуются в специальный формат, при этом извлекаются дополнительные метаданные о колонках. На втором этапе происходит генерация кандидатов и их проверка с использованием

инвертированного индекса, позволяющего эффективно находить подобные строки.

Данный алгоритм может использовать любые меры подобия, которые удовлетворяют следующим свойствам:

1. отсекать пары значений на основе упорядоченной числовой характеристики;

2. отсекать пары значений на основе неравенства их подстрок.

Второе свойство необходимо для создания инвертированного индекса, где подстроки указывают на свои родительские строки. Если две строки не имеют общих подстрок, то они считаются несхожими.

Среди других мер схожести можно выделить расстояние Хэмминга, косинусное сходство (Cosine similarity) и коэффициент Сёренсена (Dice similarity).

Далее рассматриваются основные этапы работы алгоритма.

3.3.1. Обработка данных

На первом этапе входные данные преобразуются в специальный формат и извлекаются дополнительные метаданные.

Для каждого столбца все уникальные значения группируются по их длине. В случае расстояния Левенштейна длина измеряется в количестве символов в строке, а в случае коэффициента Жаккара — в количестве токенов в строке. При этом отбрасываются колонки, которые содержат значения, длина которых более 50 символов или более 10 токенов (в случае работы с токенами). Для каждого столбца собирается информация о максимальной и минимальной длине его значений.

Алгоритмы для работы с зависимостями включения могут использоваться для работы с большими данными, из-за чего могут возникать проблемы с недостатком оперативной памяти. Для таких случаев используется контроль памяти и выгрузка колонок значений на диск. Если на этапе обработки входных данных алгоритм достигает ограничения по памяти, то алгоритм выгружает самый большой столбец на диск.

Данные на диске хранятся в отдельных файлах, сгруппированных по индексу колонки и длине значений, что позволяет считывать их без необходимости повторного чтения всей таблицы. После полной обработки входных данных удаляются дубликаты в выгруженных на диск данных, поскольку из-за раннего сброса буферов могут появляться повторения.

Также на этапе обработки входных данных происходит отсеивание неподходящих кандидатов. Например, исключаются кандидаты с пустыми столбцами и рефлексивные зависимости. В случае расстояния Левенштейна исключаются кандидаты, у которых все значения короче порогового значения. Отбрасываются также кандидаты, которые не могут быть истинными из-за слишком большой разницы в длинах значений.

После отсеивания оставшиеся кандидаты упорядочиваются: каждой ссылочной колонке сопоставляются все потенциальные колонки, которые могут от неё зависеть.

3.3.2. Поиск зависимостей

Основной принцип Sawfish заключается в том, что для отсечения кандидата `SIND` достаточно найти всего один контрпример. В отличие от зависимостей включения, в случае зависимостей включения подобия для каждого зависимого значения должно существовать хотя бы одно похожее значение (не обязательно точное) в ссылочном столбце.

Для уменьшения числа проверок Sawfish использует инвертированный индекс. Для ссылочного столбца строится инвертированный индекс, после чего для каждого значения из зависимого столбца выполняется поиск подобного значения по данному индексу.

3.3.3. Инвертированный индекс

Инвертированный индекс хранит отображение уникальных подстрок на их исходные строки. В зависимости от режима работы данные хранятся в разном виде.

- В режиме работы с токенами индекс сопоставляет токены и их исходные строки.
- В режиме работы с расстоянием Левенштейна каждая строка разбивается на сегменты, а индекс сопоставляет сегменты и их исходные строки.

В случае работы с токенами алгоритм построения индекса очевиден, поэтому далее более подробно рассматривается построение индекса в случае работы с расстоянием Левенштейна.

Сначала генерируются позиции сегментов, при этом позиции достаточно вычислить один раз для конкретного значения длины. Длина сегментов делается практически одинаковой. Если есть остаток при делении длины строки на количество сегментов, то более короткие сегменты размещаются в начале.

Подстроки извлекаются на основе сгенерированных позиций, при этом каждая подстрока сопоставляется со своей исходной строкой. Построение индекса должно происходить последовательно для возможности использования скользящего окна по длинам. Поэтому наборы сегментов хранятся отдельно и независимо друг от друга.

Важной особенностью инвертированного индекса является возможность использования скользящего окна по длинам: достаточно сравнивать только зависимые значения со значениями внутри индекса в пределах допустимого интервала. Итерация по длинам значений происходит от самых длинных к самым коротким. При переходе от одной длины к другой происходит удаление ненужного индекса и загрузка нового.

Благодаря скользящему окну нет необходимости строить весь индекс: если столбец перестаёт использоваться в качестве ссылочного, индексация прекращается досрочно, а индексы занимают меньше памяти.

3.4. Выводы

По итогам обзора выбраны подходы к обработке неточностей данных, а именно: использование алгоритмов поиска приближённых зависимостей включения и зависимостей включения подобия. Именно эти типы зависимостей включения позволяют наиболее удобно и эффективно обрабатывать данные, содержащие ошибки.

Для поддержки приближённых зависимостей включения выбрано два алгоритма:

- Spider — для поиска унарных зависимостей включения,
- Mind — для поиска n-арных зависимостей включения.

Для поиска зависимостей включения подобия принято решение использовать алгоритм Sawfish.

4. Реализация

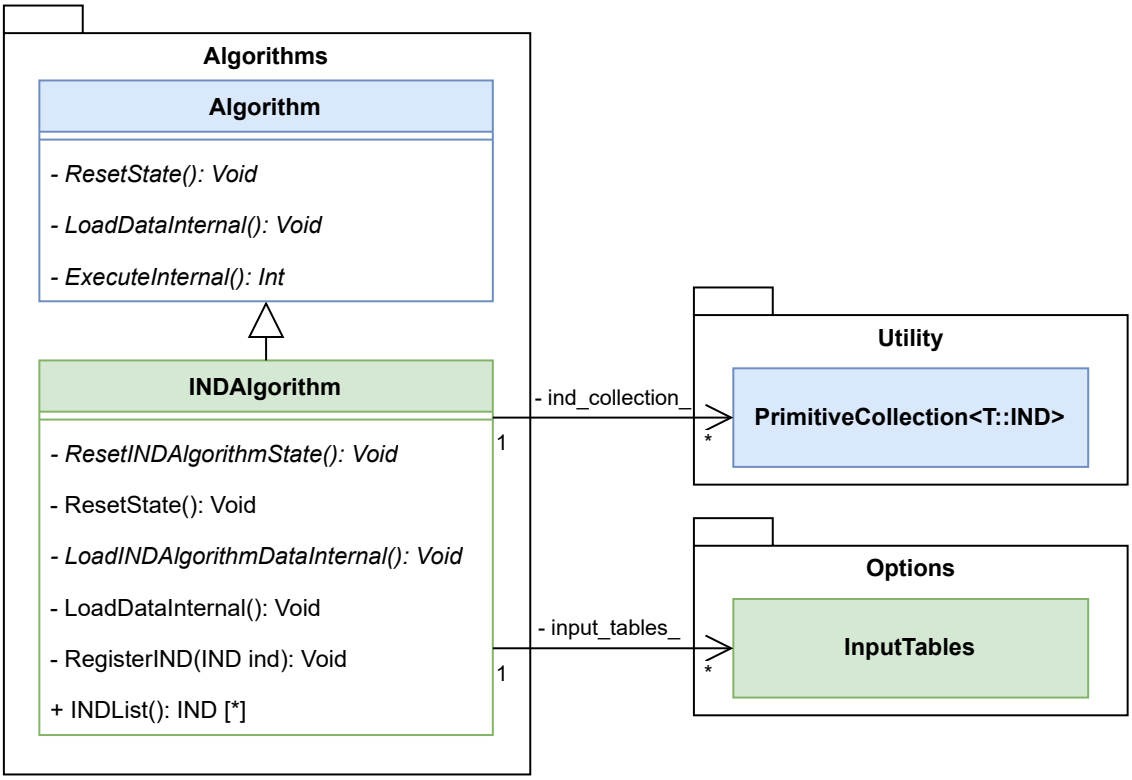

Рис. 3: Иерархия классов алгоритмов поиска IND.

На Рис. 3 представлена диаграмма, показывающая иерархию наследование классов алгоритмов поиска зависимостей включения. Зелёным цветом обозначены классы, реализованные в рамках данной работы. Наиболее важными методами при реализации нового алгоритма являются `LoadDataInternal`, `ExecuteInternal` и `ResetState`. Метод `LoadDataInternal` необходим для обработки входных таблиц, с которыми происходит дальнейшая работа. Также, поскольку загрузка данных выполняется в отдельном шаге, ещё до начала выполнения, впоследствии можно запускать один и тот же алгоритм, выставляя разные значения опций. При этом нет необходимости выполнять наиболее ресурсозатратный шаг — загрузку данных. В функции `ExecuteInternal` содержится основная логика самого алгоритма. Функция `ResetState` необходима для выполнения сброса состояния алгоритма, после выполнения которой алгоритм должен вернуться к состоянию, когда данные

были загружены. Это может быть полезно при необходимости повторного выполнения алгоритма с другими конфигурационными параметрами.

Абстрактный класс `INDAlgorithm` задаёт общий интерфейс для алгоритмов поиска IND. Он устроен аналогично другим классам, реализованным в Desbordante. Класс `INDAlgorithm` содержит метод для получения результатов работы алгоритма — списка зависимостей включения. Кроме того, он реализует метод `ResetState`, который очищает список зависимостей и вызывает абстрактный метод `ResetINDAlgorithmState`. Аналогичным образом реализован метод `LoadDataInternal`, в котором создаются схемы для входных таблиц, необходимых для регистрации зависимостей включения.

4.1. Алгоритм Spider

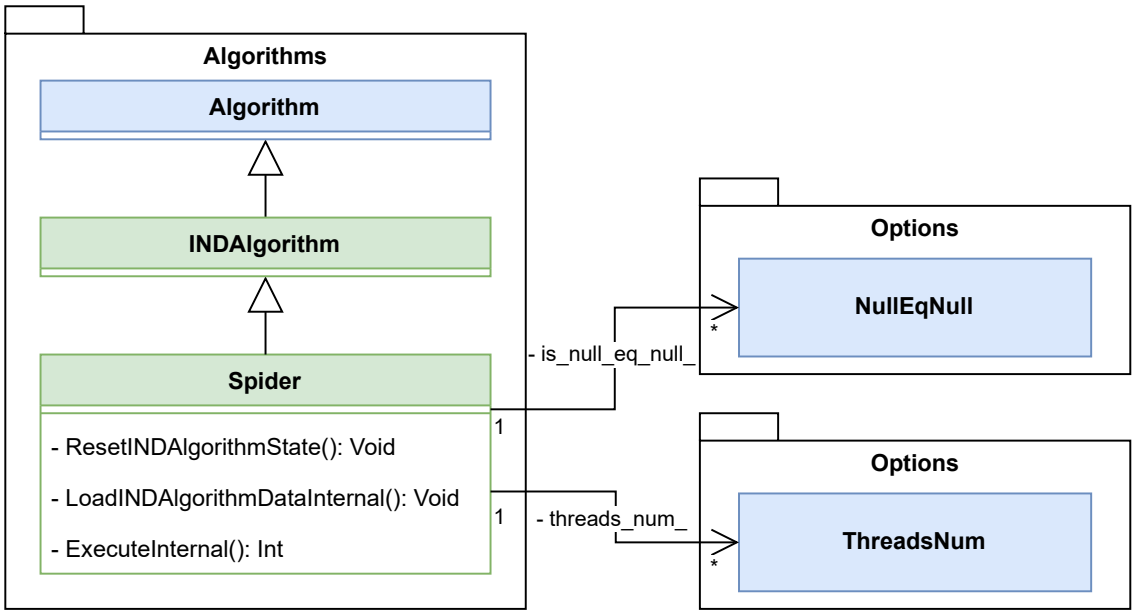

Рис. 4: Иерархия классов алгоритма Spider.

На Рис. 4 представлена диаграмма, показывающая иерархию наследования классов, реализующих алгоритмы поиска зависимостей включения. На этапе загрузки данных происходит обработка входных таблиц, по итогам которой создаются отсортированные домены для всех ко-

лонок. В методе `ExecuteInternal` содержится основная логика алгоритма, а именно генерация и проверка кандидатов. В методе `ResetState` выполняется очистка внутренних структур алгоритма, используемых на стадии выполнения.

На стадии загрузки данных SPIDER выполняет последовательную обработку входных таблиц, при этом каждая таблица обрабатывается за один проход. Для каждой таблицы создаётся набор доменов. Домен представляется в виде набора партиций. Партиция может либо находиться в оперативной памяти, либо храниться на диске. Запись партиции на диск происходит только при достижении ограничения по памяти.

Класс домена колонки инкапсулирует логику чтения с партиции, предоставляя единый интерфейс для итерации по значениям партиций. Благодаря этому можно выполнять соединение таблиц "на лету", прямо во время обработки объектов атрибутов, что позволяет избегать дополнительного взаимодействия с диском.

Создание доменов выполняется параллельно. Чтение таблицы выполняется поблочно (размер блока выбирается в зависимости от заданного пользователем параметра ограничения памяти). Блок представляет собой набор из колонок, обработка которых происходит в отдельных потоках. Таким образом, максимальная параллельность алгоритма ограничена количеством колонок в таблице.

При реализации партиций проводилось сравнение контейнеров `std::set` и `absl::btree_set`[4] для хранения значений. По результатам тестирования был выбран `absl::btree_set` благодаря лучшей производительности. Подробные результаты представлены в подразделе 5.2. В рамках данного алгоритма логичнее использовать B-дерево, поскольку на стадии заполнения доменов итераторы могут инвалидироваться. Это допустимо, так как этапы добавления значений и их чтения разделены. Таким образом, нет необходимости использовать красно-чёрное дерево, гарантирующее устойчивость итераторов.

Для подсчёта количества используемой памяти выполняется итера-

[4] https://github.com/abseil/abseil-cpp (дата обращения: 31 мая 2025 г.)

ция по всем строкам, хранящимся в партиции подсчитывается примерная оценка используемой памяти. Данная проверка дорогостоящая, поэтому она выполняется как можно реже. Для этого подсчитывается текущее среднее потребление памяти, на основе которой можно достаточно точно оценить сколько ещё блоков может быть обработано.

На этапе выполнения алгоритма Spider взаимодействует с объектами атрибутов `INDAttribute` и `AINDAttribute` для поиска точных и приближённых зависимостей (в зависимости от заданного пользователем порога ошибки). Оба класса наследуются от общего класса `Attribute`, реализующего общую логику работы с атрибутом. При этом алгоритм не работает с абстрактным базовым классом `Attribute`, поскольку классы `INDAttribute` и `AINDAttribute` предоставляют специальный интерфейс для взаимодействия. Базовый класс только реализует логику сравнения с другими атрибутами и хранит итератор домена.

Класс `INDAttribute` реализует логику объекта атрибута, представленного в главе 3.1. То есть данный объект хранит два списка индексов кандидатов и отсекает кандидатов по мере обработки атрибута. А класс `AINDAttribute` содержит только вектор счётчиков, которые подсчитывают количество совпадающих значений с каждым из кандидатов. После обработки всех объектов атрибута кандидаты не отсекаются, а валидность кандидатов определяется после обработки всех доменов (кандидат валиден, если ошибка меньше порога возможной ошибки).

Одной из возможных модификаций является стадия генерации кандидатов с их отсечением на основе максимального и минимального значения. Данная оптимизация в среднем даёт хорошее ускорение стадии проверки кандидатов, однако практически не влияет на общее время работы алгоритма. Это связано с тем, что стадия проверки кандидатов занимает в среднем около $\frac{1}{10}$ от времени выполнения стадии обработки данных. Поэтому данная модификация не использовалась в итоговом решении.

Далее перечислены особенности реализации алгоритма, которые в Metanome можно улучшить и которые были реализованы в Desbordante.

4.1.1. Обработка входных данных

В реализации от Metanome обработка колонок входных таблиц происходит последовательно в одном потоке. То есть по входной таблице с n колонками выполняется n полных проходов, где на i-ой итерации создаётся колоночный файл для i-ой колонки.

На практике, создание колоночных файлов можно выполнять одновременно, за один проход по входной таблице. При этом заполнение структур данных, в которых хранятся данные для колоночных файлов может выполняться параллельно.

В реализации от Metanome по итогам обработки очередной колонки выполняется объединение всех её партиций в один колоночный файл. Однако подобное объединение файлов может быть сделано "на лету", на стадии проверки кандидатов (в данном случае итератор должен содержать в себе логику объединения партиций).

Также, в случае, когда алгоритм не достигает ограничения по памяти, можно не создавать колоночные файлы на диске, а хранить партиции в оперативной памяти. Благодаря этому, можно избежать дорогостоящих операций ввода-вывода.

4.1.2. Поиск приближённых IND

Алгоритм Spider может быть обобщён для того, чтобы искать не только точные унарные IND, но и приближённые. Для этого объект атрибута должен для всех исходных объектов атрибутов подсчитывать количество совпадающих значений.

Важно отметить, что в случае поиска приближённых IND нельзя использовать возможность раннего завершения, из-за чего стадия проверки кандидатов может существенно замедлиться. Поэтому стоит отдельно рассматривать случай поиска точных и приближённых IND.

4.2. Алгоритм Mind

Как упоминалось ранее, алгоритм в оригинальной статье определяется в терминах реляционных баз данных. То есть, для генерации и проверки кандидатов используются SQL запросы. При этом данное решение имеет несколько недостатков:

- Некорректная работа в случае с нулевыми и пустыми значениями;
- Отсутствие возможности повторно использовать промежуточные вычисления для проверки кандидатов, из-за чего время проверки кандидатов может значительно меняться.

При реализации методов для генерации кандидатов и их проверки использовались аналогичные алгоритмы без использования SQL.

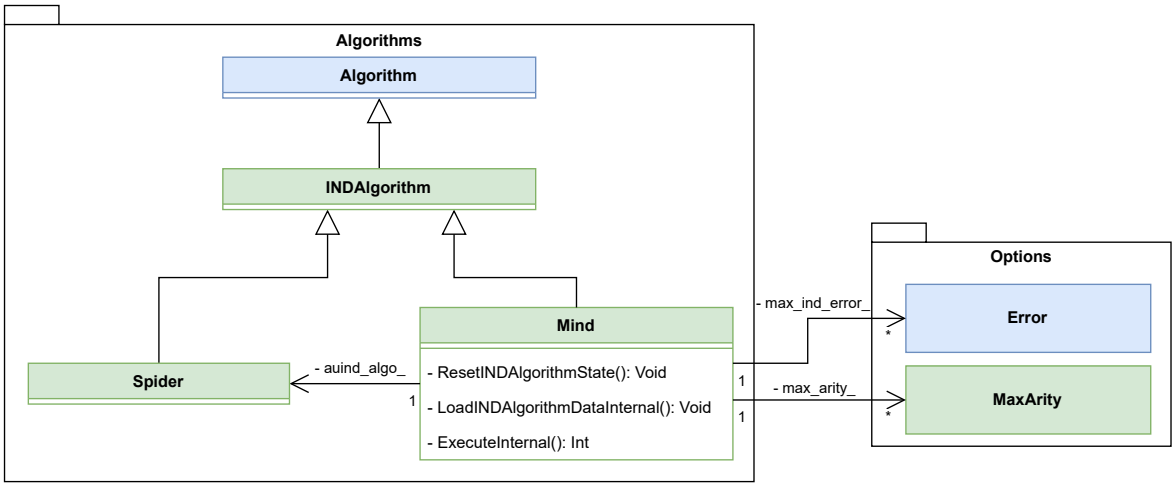

Рис. 5: Иерархия классов для алгоритма Mind.

На Рис. 5 представлена иерархия классов для алгоритма Mind. Алгоритм Mind использует алгоритм Spider для поиска унарных зависимостей включения, а для поиска зависимостей большей размерности применяется непосредственно алгоритм Mind. На каждом уровне выполняется генерация кандидатов, после чего происходит их проверка. Алгоритм генерации кандидатов является хорошо известным и не будет рассматриваться далее.

Наибольший интерес представляет реализация проверки валидности кандидатов. Алгоритм отличается в случае проверки точной зави-

симости включения и приближенной. На Листинге 6 представлен алгоритм для проверки точных зависимостей:

```
1  Input: d a database, I — IND candidate.
2  Output: true, if I holds, false otherwise.
3
4  lhs_stream := get_projected_stream(d, I.lhs.table, I.lhs.indices)
5  rhs_stream := get_projected_stream(d, I.rhs.table, I.rhs.indices)
6  rhs_hash_set := create_hash_set(rhs_stream)
7
8  forall lhs_row ∈ lhs_stream do
9      if lhs_row ∉ rhs_hash_set then
10         return false
11     end if
12 end forall
13
14 return true
```

Листинг 6: Алгоритм проверки кандидата IND

Сначала выполняется создание хэш таблицы для проекции правого отношения на $I.rhs.indices$. После чего выполняется проход по всем кортежам проекции левого отношения на $I.lhs.indices$.

На Листинге 7 представлен алгоритм для проверки приближенных зависимостей:

```
1  Input: d a database, I — IND candidate, ε — error threshold.
2  Output: (holds, error), where holds is true iff I holds.
3
4  lhs_stream := get_projected_stream(d, I.lhs.table, I.lhs.indices)
5  rhs_stream := get_projected_stream(d, I.rhs.table, I.rhs.indices)
6  rhs_hash_set := create_hash_set(rhs_stream)
7  lhs_hash_set := create_hash_set(lhs_stream)
8
9  lhs_cardinality := cardinality(lhs_hash_set)
10 disqualify_row_limit := ⌊lhs_cardinality * ε⌋ + 1
```

```
11 disqualify_row_count := 0
12
13 forall lhs_row ∈ lhs_hash_set do
14     if lhs_row ∉ rhs_hash_set then
15         disqualify_row_count := disqualify_row_count + 1
16         if disqualify_row_count == disqualify_row_limit then
17             return (false, None)
18         end if
19     end if
20 end forall
21
22 error := disqualify_row_count/lhs_cardinality
23 if error ≤ ε then
24     return (true, error)
25 else
26     return (false, None)
27 end if
```

Листинг 7: Алгоритм проверки кандидата AIND

В отличие от оригинального алгоритма, здесь используется раннее терминирование в случае, когда количество ошибок достигает лимита. Также, при проверке приближенной зависимости необходимо подсчитывать мощность левого спроецированного множества. Из-за чего хэш-таблица создается не только для правого отношения, а также и для левого.

Дополнительно, для данного алгоритма была реализована модификация, позволяющая повторно использовать ранее построенную хэш-таблицу для левой части зависимости. Поскольку кандидаты на проверку сортируются по возрастанию левой части (lhs), часто подряд проверяются зависимости с одинаковой левой стороной. В этом случае нет необходимости повторно строить хэш-таблицу для левой части: вместо этого используется кэшированное значение, что позволяет существенно сократить количество лишних операций.

Эта оптимизация не изменяет логику алгоритма, но значительно улучшает производительность. Соответствующие экспериментальные результаты представлены в разделе 5.3.

В рамках работы реализован верификатор AIND на основе общей логики проверки кандидатов. Далее показано, как он интегрируется в процесс анализа и используется совместно с алгоритмом MIND.

4.2.1. Алгоритм INDVerifier

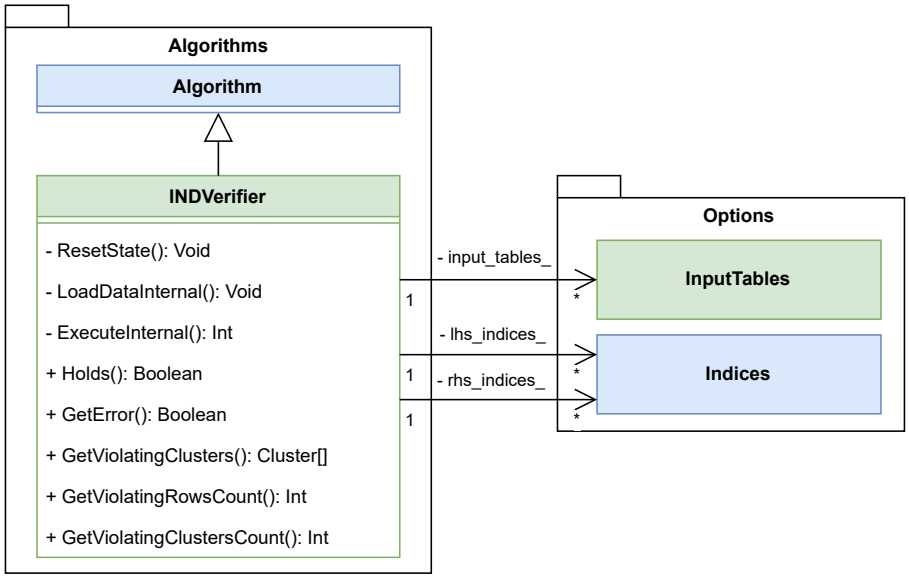

Рис. 6: Иерархия классов для алгоритма INDVerifier.

Иерархия классов представлена на Рис. 6. Алгоритм принимает одну/две таблицы, а также индексы левой части зависимости и индексы для правой части. Если указана одна таблица, то зависимость проверяется в рамках одного отношения, в противном случае $lhs_indices$ относятся к первой таблице, а $rhs_indices$ — ко второй.

Для реализации алгоритма верификации IND используется схожий алгоритм проверки кандидатов, как и в алгоритме MIND, о котором говорилось ранее. Однако, в данном случае также происходит сбор статистики о кластерах, на которых зависимость не выполняется. Верификатор возвращает список из кластеров, где кластер (т.е. набор индексов) соответствует одному уникальному значению в левой части отношения.

Для пользователя наибольший интерес представляют те зависимости, на которых зависимость практически полностью удерживается, то есть для которых верификатор находит мало кластеров.

Алгоритм INDVerifier реализован на языке C++, а также добавлен интерфейс к нему для взаимодействия через Python, подобно аналогичным алгоритмам верификации в Desbordante.

Пример совместного использования алгоритмов. Рассмотрим пример использования пайплайна из алгоритма для поиска приближенных зависимостей включения и алгоритма верификации.

id	customer_id	product
1	101	Laptop
2	102	Phone
3	103	Tablet
4	104	Monitor
5	108	Keyboard
6	201	Mouse
7	102	Charger

Таблица 3: Orders

id	name	country
101	Alice	USA
102	Bob	UK
103	Charlie	Canada
104	David	Germany
105	Eve	France

Таблица 4: Customers

Для демонстрации примера используются таблицы *orders* 3 и *customers* 4. Рассмотрим скрипт для построения пайплайна на Рис. 8, написанный на языке Python с использованием библиотеки *desbordante*:

```
1  import desbordante
2  import pandas as pd
3
4  def print_clusters(algo, table, indices):
5      print("Number of clusters violating IND:",
6            f"{len(algo.get_violating_clusters())}")
7      for i, cluster in enumerate(algo.get_violating_clusters(),
8                                   start=1):
9          print(f"#{i} cluster:")
10         for el in cluster:
11             values = " ".join([f"{table[table.columns[idx]][el]}"
```

```
                    for idx in indices])
            print(f"{el}: {values}")

TABLE_NAMES = ['orders.csv', 'customers.csv']
TABLES = [pd.read_csv(name) for name in TABLE_NAMES]
ERROR = 0.4

print(f"Run AIND algorithm with error threshold {ERROR}")
algo = desbordante.aind.algorithms.Default()
algo.load_data(tables=TABLES)
algo.execute(error=ERROR)

print("List of found AINDs:")
inds = algo.get_inds()
ainds = list(filter(lambda i: i.get_error() != 0.0, inds))
for aind in ainds:
    print(aind)

print("Run AIND Verifier for each AIND:")
for aind in ainds:
    (lhs_col, lhs_indices) = aind.get_lhs().to_index_tuple()
    (rhs_col, rhs_indices) = aind.get_rhs().to_index_tuple()
    lhs_table = TABLES[lhs_col]
    rhs_table = TABLES[rhs_col]

    verifier = desbordante.aind_verification.algorithms.Default()
    verifier.load_data(tables=[lhs_table, rhs_table])
    verifier.execute(lhs_indices=lhs_indices,
                     rhs_indices=rhs_indices)
    print("AIND:", aind)
    print_clusters(verifier, lhs_table, lhs_indices)
```

Листинг 8: Скрипт для совместно использования алгоритмов

В рамках пайплайна сначала происходит выполнение алгоритма для поиска приближенных зависимостей (строки 20-22), а также для каждой приближенной зависимости запускается алгоритм верификации с выводом информации о кластерах (строки 31-43), на которых зависимость нарушается. На Листинге 9 представлен вывод.

```
1  Run AIND algorithm with error threshold 0.4
2  List of found AINDs:
3  (orders, [customer_id]) -> (customers, [id]) with error = 0.33
4  (customers, [id]) -> (orders, [customer_id]) with error = 0.2
5
6  Run AIND Verifier for each AIND
7  AIND: (orders, [customer_id]) -> (customers, [id]) with error =
     0.33
8  Number of clusters violating IND: 2
9  #1 cluster:
10 5: 201
11 #2 cluster:
12 4: 108
13
14 AIND: (customers, [id]) -> (orders, [customer_id]) with error =
     0.2
15 Number of clusters violating IND: 1
16 #1 cluster:
17 4: 105
```

Листинг 9: Вывод скрипта при выполнении AIND алгоритма

Алгоритму поиска приближенных зависимостей удалось найти две приближенные зависимости. Вторая зависимость не представляет интереса, поскольку $customers.id$ является первичным ключом. Первая же зависимость должна удерживаться, поскольку в данном случае $orders.customer_id$ является ссылкой на первичный ключ $customers.id$. Очевидно, что пятый и четвертый кортежи являются некорректными, поскольку они ссылаются на некорректный индекс.

4.3. Алгоритм Sawfish

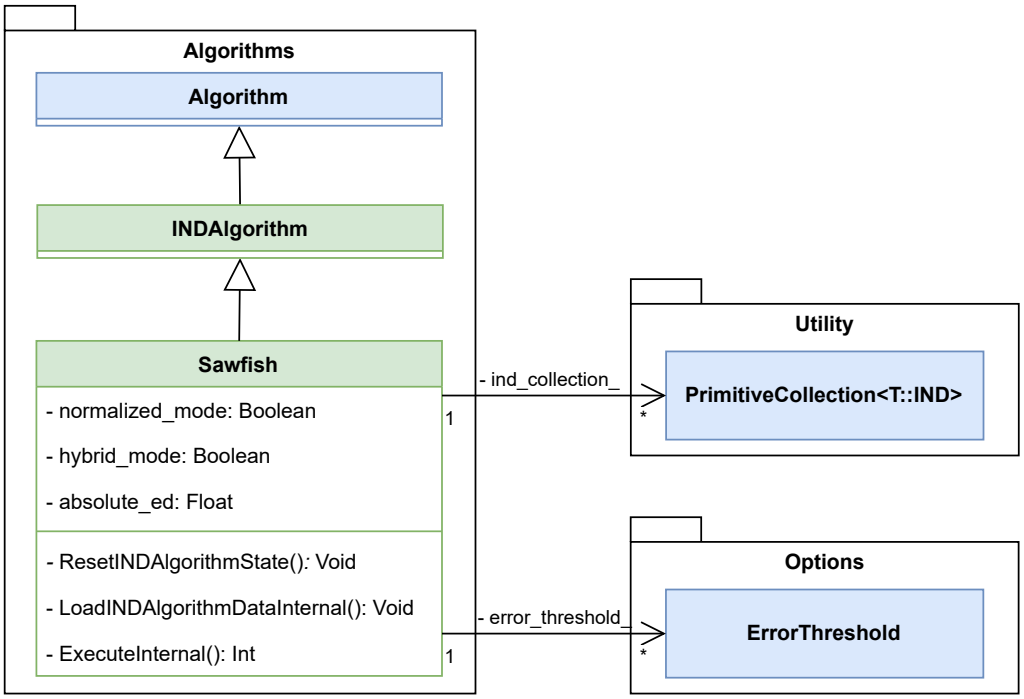

Рис. 7: Иерархия классов для алгоритма Sawfish.

На Рис. 7 представлено место алгоритма Sawfish в иерархии классов, используемой в Desbordante. На этапе загрузки данных выполняется начальная трансформация входных таблиц. Все уникальные значения в колонках извлекаются и группируются по их длине. Каждому значению длины сопоставляется хэш-таблица из значений. В реализации на базе Metanome для данных значений используется специальный класс `SubstringableString`. Этот класс предоставляет методы для эффективной работы с подстроками и применяется во всех методах алгоритма, где необходимо производить сравнения строк или их подстрок. Важно отметить, что данный класс используется как для хранения самих значений, так и для подстрок.

При реализации хранения значений в Desbordante было принято иное архитектурное решение: использовать класс `std::string` для значений в сочетании с `std::string_view` там, где это необходимо (при сравнении подстрок). Это позволяет улучшить производительность и читаемость кода.

По аналогии с реализацией, представленной в Metanome, для подсчёта значений метрики схожести используется класс `SimilarityMeasureManager`. Этот класс инкапсулирует логику сравнения значений и поддерживает несколько режимов работы.

- Нормализованный режим: используется для получения значения схожести в пределах от 0 до 1, что удобно для универсальной обработки результатов и последующего сравнения.

- Абсолютный режим: активируется при указании максимальной допустимой длины различий между строками — полезен в случаях, когда метрика должна соответствовать определённым числовым ограничениям.

- Относительный режим: применяется, когда задан порог допустимой ошибки, позволяя игнорировать незначительные расхождения.

- Гибридный режим: совмещает элементы абсолютного и относительного подходов, что может быть полезно для адаптации под различные типы данных и условий задачи.

Класс обеспечивает гибкий интерфейс для интеграции с остальной частью алгоритма, позволяя варьировать поведение подсчёта схожести без необходимости в изменении основной логики обработки данных.

В рамках данной работы отдельное внимание уделялось стадии обработки входных данных и построению чанков для каждого значения длины. Ключевым моментом при реализации данной логики стало использование подходящей структуры данных для хранения уникальных значений — хэш-таблицы. Поскольку операция вставки в хэш-таблицу является базовой в процессе построения чанков, производительность этой структуры оказывает значительное влияние на общее время работы алгоритма.

В рамках исследования в разделе 5.4 были протестированы несколько реализаций хэш-таблиц:

- стандартная библиотечная реализация `std::unordered_set`;
- хэш-таблицы из проекта emhash[5], позиционируемые как высокопроизводительные контейнеры;
- хэш-таблица `absl::flat_hash_set` из библиотеки Abseil, оптимизированная для высокой скорости и низкого потребления памяти.

По результатам экспериментального сравнения, реализация `absl::flat_hash_set` показала наилучшую производительность. Она значительно опережала `std::unordered_set` как по времени вставки, так и по объёму потребляемой памяти, а также опережала альтернативные реализации от emhash по стабильности и скорости в условиях высоких нагрузок. В связи с этим именно `absl::flat_hash_set` была выбрана в качестве основной структуры хранения уникальных значений при построении чанков.

Дополнительно стоит отметить, что этап трансформации данных потенциально может быть ускорен за счёт параллельного чтения и трансформации. Такой подход позволит более эффективно использовать ресурсы многоядерных систем. При этом вставку в хэш-таблицы можно выполнять параллельно (где параллельность ограничена количеством колонок в таблице), поскольку хэш-таблицы независимы друг от друга. Аналогичная модификация использовалась в рамках алгоритма Spider и описывалась в подразделе 4.1.

4.4. Итоги реализации

По итогам данной работы алгоритмы Spider, Mind и INDVerifier были полностью реализованы и интегрированы в рамках платформы Desbordante. Для каждого алгоритма выполнено тестирование и созданы Python-обвязки. Отдельные модификации алгоритмов (использование B-дерева в случае алгоритма Spider и кеширование хэш-таблиц в случае алгоритма Mind, например), а также поддержка алгоритма Sawfish находятся на стадии внедрения в систему.

[5] https://github.com/ktprime/emhash (дата обращения: 31 мая 2025 г.)

5. Эксперименты

В данной главе представлены результаты экспериментального исследования реализованных алгоритмов поиска приближённых зависимостей включения и зависимостей включения подобия. В экспериментах участвуют три алгоритма: Spider, Mind и Sawfish, каждый из которых решает свою задачу по поиску зависимостей включения различного типа.

Разделение экспериментов по задачам.

- Сравнение модифицированной реализации алгоритмов Spider с реализацией в Metanome.
- Исследование производительности алгоритма Mind, реализованного в рамках данного исследования, без сравнения с Metanome из-за несовместимости с форматом данных и отсутствия поддержки приближённых зависимостей включения.
- Сравнение модифицированной реализации алгоритма Sawfish с реализацией в Metanome.

Более подробно детали проведения экспериментов, включая описание используемых наборов данных, приведены далее в соответствующих подразделах. При этом стоит отметить, что для алгоритмов, работающих с приближёнными зависимостями включения, и для алгоритма, реализующего поиск зависимостей включения подобия, используются разные наборы данных, учитывающие специфику задачи.

5.1. Методология

Для оценки производительности алгоритмов в качестве ключевой метрики используется время выполнения. В экспериментах рассматривается среднее время выполнения за 10 запусков. Перед каждым запуском производилась очистка файлового кеша командой `/proc/sys/vm/drop_caches`, чтобы минимизировать влияние кэширования на результаты. На протяжении выполнения всех экспериментов

файл подкачки был отключен с помощью команды `swapoff -a`. Во время выполнения экспериментов все ресурсоёмкие процессы были отключены и фиксировалась частота процессора.

Характеристики системы. Для экспериментов использовалась система со следующими характеристиками: AMD Ryzen 7 4800H CPU @2.90GHz x 8 cores (16 threads), 32 GiB RAM, SSD A-Data S11 Pro AGAMMIXS11P512GT-C 512GB, running Ubuntu 24.04.1.

Конфигурация Java, используемая Metanome: openjdk 21.0.7 2025-04-15, OpenJDK Runtime Environment (21.0.7+6-Ubuntu-0ubuntu124.04). Конфигурация, используемая Desbordante: gcc (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0, ldd (Ubuntu GLIBC 2.39-0ubuntu8.4) 2.39. Компиляция Desbordante происходила с опцией `-O3`.

5.2. Алгоритм Spider

5.2.1. Наборы данных

Имя	Размер	Тип	Таблицы	Атрибуты	Строки	unary IND
Brazilian E-Commerce[6]	126.3 МБ	Реальный	9	52	1.6 M	23
FitBit Fitness Tracker Data	330 МБ	Реальный	18	259	8.1 M	8798
TPC-H (SF=1, 2, 4, 8)	SF · 1.1 ГБ	Синтетический	8	61	SF · 8.7 млн	96 (97 при SF=8)
haINDgen (generator)	862 МБ	Синтетический	2	36	6.1 M	18
CIPublicHighway	28.3 МБ	Реальный	1	18	0.4 M	65

Таблица 5: Информация о наборах данных для экспериментов

В экспериментах используется три реальных набора данных из различных областей и 3 синтетических, информация о которых представлена в таблице 5. Главной целью было собрать набор данных, каждый из которых будет обладать особенностями. `Brazilian E-Commerce` (ECOMM) — это реальный набор данных о продажах, который содержит довольно длинные строки с отзывами пользователей. `FitBit Fitness Tracker Data` (FITA) — это широкий набор данных, который содержит большое количество зависимостей. `CIPublicHighway` (CI_PH) — это разреженная таблица с большим процентов пустых значений. `TPC-H` — широко используемый синтетический набор данных, для которого далее расматриваются коэффициенты масштаба от 1 до 8. Последний на-

бор данных `haINDgen` (HAIND) генерируется утилитой, которая может создавать данные с n-арным числом IND, где n велико.

5.2.2. Экспериментальное исследование

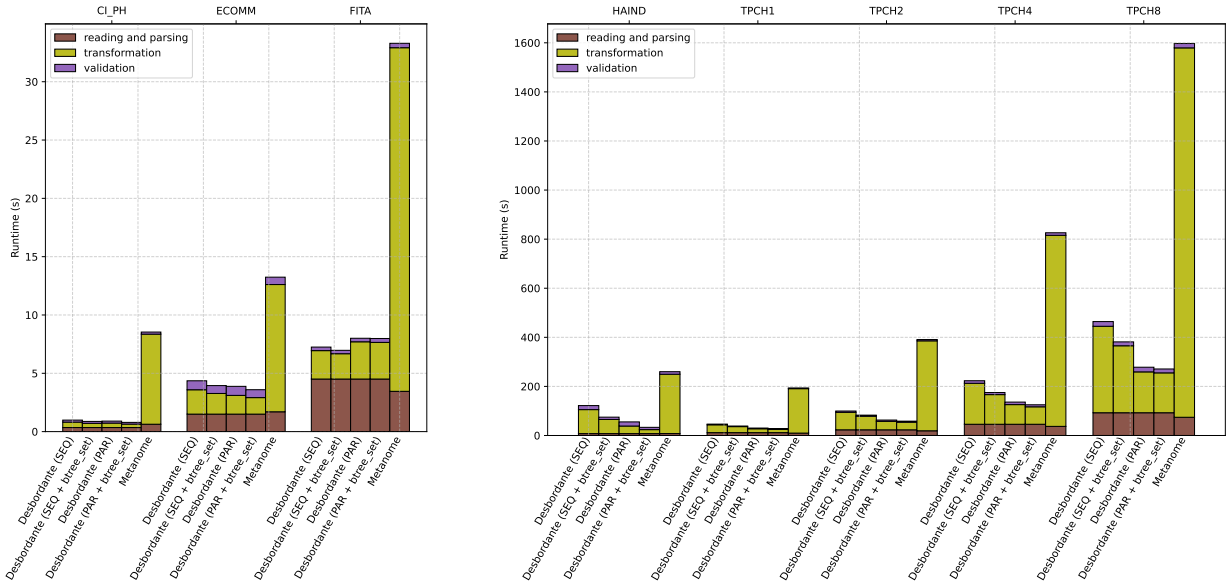

Рис. 8: Время выполнения Spider для Metanome и Desbordante.

На Рис. 8 представлена диаграмма сравнения времени выполнения нескольких версий алгоритма Spider из Desbordante с однопоточной версией из Metanome. В частности, в Desbordante показаны: базовая версия без основных модификаций (SEQ, последовательная), версия с использованием `absl::btree_set` вместо `std::set` для хранения домена значений, версия с параллельной загрузкой данных в эти домены (PAR), которая запускалась в 8 потоках, а также версия, включающая оба улучшения.

На диаграмме каждый сегмент столбца соответствует определённой стадии выполнения алгоритма.

- Первый сегмент отражает время, затрачиваемое на чтение данных с диска и обработку значений `csv`-таблицы — этап подготовки значений перед вставкой в домен значений.

- Второй сегмент показывает время, необходимое для трансформации — заполнению доменов значений. Вместе с чтением эти

два этапа составляют стадию обработки данных, которая упоминалась в обзоре алгоритма в подразделе 3.1. При этом чтение и трансформация не являются отдельными стадиями, а трансформация происходит одновременно с чтением данных.

- Третий сегмент соответствует этапам генерации кандидатов и проверки кандидатов. Отдельный сегмент для генерации кандидатов не выделяется, поскольку этот этап практически не занимает времени.

Разделение стадии обработки входных данных на отдельные сегменты сделано намеренно. Этап чтения технически находится вне основной области ответственности алгоритма и может быть опущен при сравнении производительности реализаций. Это важно, поскольку если одна система быстрее выполняет чтение значений, то она может продемонстрировать лучшее общее время, хотя сам алгоритм при этом может работать медленнее. В случае экспериментов, представленных на Рис. 8, этап чтения данных в некоторых случаях быстрее в реализации Desbordante, в других — в реализации Metanome. Основное внимание при анализе будет уделяться времени выполнения без учёта стадии чтения, однако для полноты картины приводятся значения общего времени выполнения.

Dataset	**SEQ**	**SEQ +** `btree_set`	**PAR(8)**	**PAR(8) +** `btree_set`
CIPublicHighway	8.65 (12.43)	9.77 (15.11)	9.48 (14.38)	10.79 (17.96)
E-Commerce	3.04 (4.04)	3.36 (4.72)	3.41 (4.85)	3.69 (5.51)
FitBit Data	4.59 (10.89)	4.78 (12.13)	4.16 (8.54)	4.17 (8.59)
HaINDgen	2.13 (2.21)	3.48 (3.76)	4.70 (5.29)	7.83 (9.89)
TPC-H (SF=1)	4.18 (5.29)	5.03 (6.83)	6.39 (9.83)	7.04 (11.56)
TPC-H (SF=2)	3.92 (4.86)	4.72 (6.22)	6.21 (9.35)	6.69 (10.54)
TPC-H (SF=4)	3.70 (4.45)	4.72 (6.10)	6.07 (8.73)	6.60 (9.93)
TPC-H (SF=8)	3.44 (4.10)	4.19 (5.28)	5.74 (8.21)	5.89 (8.54)

Таблица 6: Ускорение модификаций Desbordante относительно Metanome. Значения указаны как с учётом времени чтения данных, так и без него (в скобках).

В таблице 6 представлены коэффициенты ускорения различных мо-

дификаций алгоритма Spider в Desbordante по сравнению с версией из Metanome. В среднем ускорение стандартной версии достигает порядка 5 раз, при этом для большинства наборов данных оно находится в диапазоне от 5 до 6 раз.

Следует отметить, что для относительно небольших наборов данных параллельная загрузка доменов значений не даёт существенного выигрыша из-за накладных расходов на подготовку блоков данных перед параллельной обработкой и запуском потоков для обработки. Однако для больших наборов данных этот подход обеспечивает заметный прирост производительности.

Наибольший прирост даёт параллельное заполнение структур доменов значений, что объясняется значительной долей времени, затрачиваемой на этот этап. Кроме того, заметный выигрыш в производительности обеспечивается отсутствием принудительной записи колоночных файлов на диск после обработки данных.

Использование контейнера `absl::btree_set` вместо стандартного `std::set` дало заметное ускорение, варьирующееся от примерно 10% до более 150% по сравнению с последовательной версией без модификаций. Это связано с особенностями реализации B-деревьев в библиотеке Abseil. В отличие от классических сбалансированных бинарных деревьев, таких как красно-чёрные деревья, используемых в `std::set`. Таким образом, данная замена позволила существенно сократить накладные расходы на операции с доменами значений.

В случае одновременного применения обеих модификаций — параллельной загрузки и замены контейнера на `absl::btree_set` — ускорение в среднем составляет около 10.3 раз без учёта времени чтения данных и примерно в 6.6 раз с учётом общего времени.

5.3. Алгоритм Mind

В экспериментах для алгоритма Mind используются два набора данных (`CIPublicHighway` и `TPC-H` с коэффициентом масштаба 1), которые были описаны при тестировании алгоритма Spider в предыдущем под-

разделе 5.2. В дальнейшем будет рассматриваться только стадия поиска n-арных приближённых зависимостей включения, поскольку для поиска унарных зависимостей используется алгоритм Spider. Также конфигурации алгоритма максимальная размерность зависимости была установлена равной двум.

Для данного алгоритма не представляется возможным использовать остальные наборы данных, выбранных для проведения экспериментов, поскольку поиск приближённых зависимостей включения занимает значительно больше времени (на порядок дольше). Проверка всех кандидатов в случае наборов данных FITA, HAIND и TPC-H с коэффициентом масштаба больше единицы оказывается чрезмерно затратной по времени. Это один из существенных недостатков алгоритма, поскольку поиск подобных зависимостей является очень дорогим.

Далее рассматривается наиболее важная модификация — кеширование хэш-таблицы, позволяющее избежать дорогостоящего её пересоздания, которая описывается в подразделе 4.2.

Порог ошибки	0.000	0.001	0.005	0.010	0.020	0.040
Без кеширования	402	597	583	656	780	1,413
С кешированием	403	351	352	408	474	856

Таблица 7: Сравнение времени вычисления (в секундах) между предыдущей и модифицированной реализацией алгоритма на наборе данных CIPublicHighway при различных порогах погрешности

В таблице 7 представлены результаты сравнения времени поиска зависимостей размерности 2 на наборе данных `CIPublicHighway`. Можно заметить, что при значении порога ошибки 0.001 все зависимости удаётся найти быстрее при использовании кеширования, чем без кеширования. Это связано с тем, что в данном случае построение хэш-таблицы оказывается менее затратным, чем многократно итерироваться по исходным значениям. Это наиболее интересный результат, поскольку обычно ожидается, что поиск точных зависимостей всегда должен выполняться быстрее, чем поиск приближённых. Также заметно, что кеширование существенно сокращает время выполнения, посколь-

ку в данном случае значительная часть кандидатов имеют одинаковую комбинацию колонок в зависимой части (в среднем каждые пять кандидатов имеют одну и ту же левую часть).

Порог ошибки	0.000	0.001	0.005	0.010	0.020	0.040
Без кеширования	478	1,274	1,911	1,939	2,276	2,554
С кешированием	481	955	1,502	1,522	1,864	2,117

Таблица 8: Сравнение времени вычисления (в секундах) между предыдущей и модифицированной реализацией алгоритма на наборе данных TPC-H (SF=1) при различных порогах погрешности

В таблице 8 представлены аналогичные результаты для `TPC-H` (SF=1). В этом случае результаты оказываются более предсказуемыми, так как объём данных значительно больше, что делает построение хэш-таблицы для левой части чрезмерно затратным и приводит к увеличению времени выполнения примерно втрое без кеширования и вдвое — с кешированием (в отличие от предыдущего набора данных, здесь в среднем только два кандидата имеют одну и ту же левую часть).

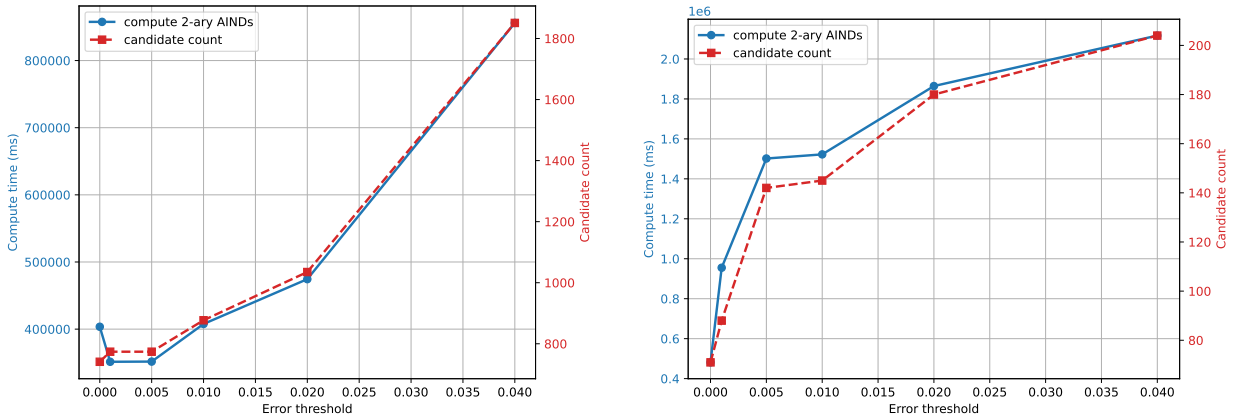

Рис. 9: Сравнение времени выполнения алгоритма MIND на двух наборах данных (CIPublicHighway и TPC-H, соответственно) в зависимости от максимального порога ошибки.

На Рис. 9 наглядно показана зависимость времени выполнения модифицированной версии алгоритма от количества кандидатов. Зависимость является полностью линейной, за исключением особенности в случае CIPublicHighway, которая наблюдается при минимальных порогах ошибки, обсуждённых ранее.

Подводя итоги, алгоритм MIND демонстрирует ожидаемое поведение, но в данном виде он пригоден только для работы с небольшими объёмами данных.

5.4. Алгоритм Sawfish

5.4.1. Наборы данных

Имя	Размер	Тип	Таблицы	Атрибуты	Строки
CENSUS	126.3 МБ	Реальный	1	42	199 тыс
WIKIPEDIA	302 МБ	Реальный	2	12	8.1 млн
TPC-H (SF=1...16)	SF · 1.1 ГБ	Синтетический	8	61	SF · 8.7 млн
IMDB	5.7 ГБ	Реальный	22	94	761 млн

Таблица 9: Информация о наборах данных для экспериментов

Для тестирования алгоритма SAWFISH используются те же четыре публично доступных набора данных[7], что и в [11], включая три реальных (`CENSUS`, `WIKIPEDIA` и `IMDB`) и один синтетический (`TPC-H`). В исходной работе все таблицы одного набора объединялись в одну. В данной работе таблицы, напротив, сохраняются раздельно и представлены в виде отдельных `csv`-файлов, чтобы уменьшить общий объём данных и избежать большого количества пропущенных значений, возникающих при объединении таблиц с разным количеством строк.

5.4.2. Экспериментальное исследование

На основе этих данных было проведено экспериментальное исследование, в котором основное внимание сосредоточено на стадии обработки входных данных. Этапы генерации кандидатов и проверки зависимостей в рамках этого исследования почти не анализируются. Это связано с тем, что на практике именно стадия обработки данных чаще всего является наиболее времязатратной. Тем не менее, в некоторых конфигурациях параметров и на специфических данных вклад стадии проверки может быть существенным, что подробно обсуждается в работе [11], в которой и был предложен данный алгоритм.

[7]https://hpi.de/naumann/projects/repeatability/data-profiling/metanome-ind-algorithms.html

Числовые колонки исключаются из рассмотрения, так как зависимости между ними на пракике редко оказываются полезными и зачастую представляют собой шум. Порог Левенштейна будет установлен в ноль, то есть будут рассматриваться только такие зависимости, где значения совпадают в точности.

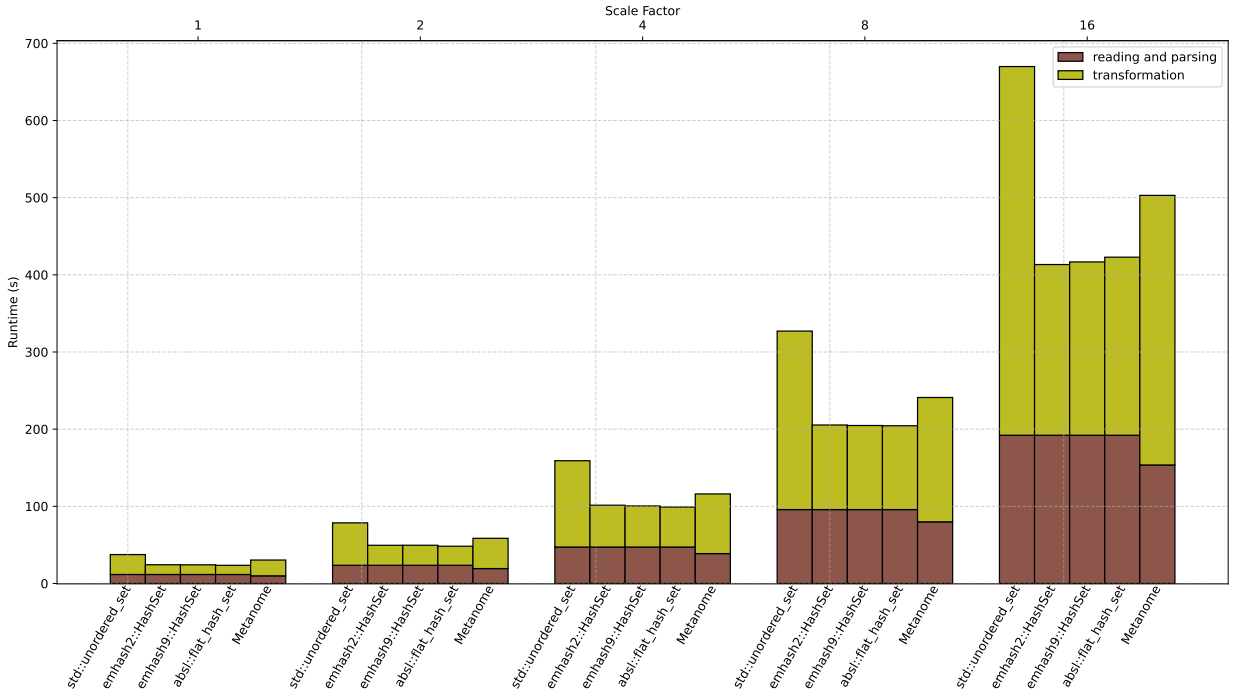

Рис. 10: Время обработки входных данных (TPC-H) для алгоритма Sawfish в зависимости от коэффициента масштаба.

На Рис. 10 представлено сравнение времени обработки данных для разных реализаций в случае Desbordante с реализацией от Metanome. На этапе обработки данных можно выделить две основных задачи подобно подходу, использованному при тестировании алгоритма Spider в подразделе 5.2:

- чтение данных с диска и обработка значений;
- преобразование данных в специальный формат.

В отличие от набора данных, использовавшегося для тестирования алгоритмов, работающих с приближёнными зависимостями включения, для предложенных наборов данных в случае Metanome чтение данных с диска и обработка значений выполняется немного быстрее.

В то время как задача чтения данных также находится за пределами данного алгоритма, преобразования входных данных в колоночные представления является здесь наиболее важной. От выбора реализации хэш-таблицы, в которой будет храниться колоночное представление, сильно зависит итоговое время работы. В таблице 10 наиболее плохие результаты показывает стандартная реализация хэш-таблицы `std::unordered_set`. Использование сторонних хеш-таблиц позволяет получить значительный прирост.

	SF 1	SF 2	SF 4	SF 8	SF 16
std::unordered_set	25.8	55.0	111.8	231.3	477.7
emhash2::HashSet	12.6	25.9	54.2	109.6	221.2
emhash9::HashSet	12.5	26.0	53.3	108.9	224.5
absl::flat_hash_set	11.9	24.7	51.9	108.6	230.7
metanome	20.5	39.2	77.3	161.1	349.3

Таблица 10: Время преобразования (в секундах) для разных реализаций хэш-таблиц в зависимости от коэффициента масштаба TPC-H

Можно заметить, что время преобразования растёт линейно в зависимости от коэффициента масштаба. Наихудшие результаты показывает хэш-таблица из стандартной библиотеки. При этом в среднем наилучшее время показывает `absl::flat_hash_set`, а именно в среднем трансформация работает в 1.5–1.7 раза быстрее. А `emhash2` и `emhash9` хэш-таблицы показывают всегда сопоставимые значения с реализацией от `absl`.

На Рис. 11 представлена диаграмма со сравнением времени выполнения между реализациями алгоритма Sawfish в Desbordante и Metanome.

	CENSUS	WIKIPEDIA	IMDB	TPCH1	TPCH2	TPCH4	TPCH8	TPCH16
Ускорение относительно Metanome	1.93 (4.28)	1.27 (2.12)	1.10 (1.98)	1.75 (2.66)	1.74 (2.61)	1.63 (2.40)	1.61 (2.31)	1.54 (2.14)

Таблица 11: Ускорение модификаций Desbordante относительно Metanome с учётом времени чтения значений и без (в скобках).

В таблице 6 представлено соотношение времени выполнения между реализациями. Значительное ускорение на `CENSUS` обусловлено специ-

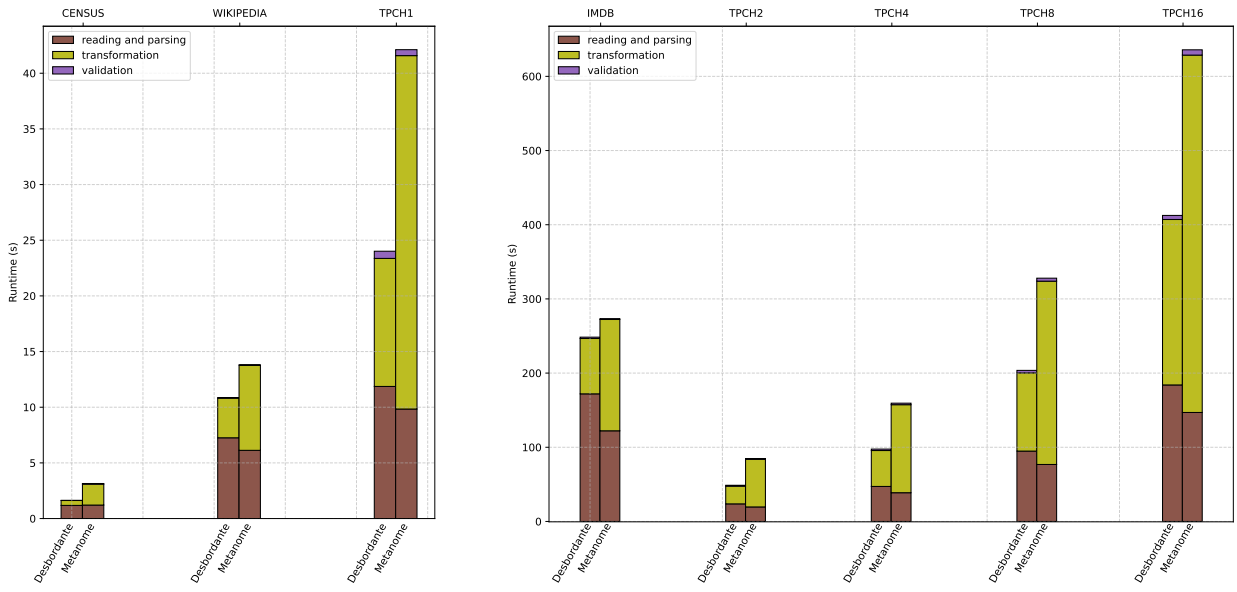

Рис. 11: Время выполнения Sawfish для Metanome и Desbordante.

фическими особенностями Java-реализации, включая затраты на финальную очистку памяти (большую часть времени занимает очистка колонок на удаление). Остальные наборы данных демонстрируют устойчивое ускорение около 2.0-2.5.

Показательно, что даже на крупных и сложных наборах данных, таких как `IMDB` и `TPC-H` (при любых значениях коэффициента масштаба), ускорение (без учёта чтения данных) сохраняется в тех же пределах.

Обобщая представленные результаты, можно заключить, что Desbordante демонстрирует значительное преимущество по времени выполнения по сравнению с Metanome в рамках алгоритма Sawfish на всех протестированных наборах данных.

Заключение

По итогам работы были получены следующие результаты.

- Выполнен обзор предметной области и выбраны расширения зависимостей включения для обработки неточностей в данных (AIND, SIND).
- Реализованы алгоритмы для работы с расширениями зависимостей включения, а именно:
  - алгоритм поиска унарных зависимостей Spider и обобщён для поиска приближённых зависимостей;
  - алгоритм поиска приближенных зависимостей Mind;
  - алгоритм верификации приближённых зависимостей включения INDVerifier и алгоритм для создания пайплайнов;
  - выполнено тестирование и добавлены интерфейсы для алгоритмов для взаимодействия через Python.
- Выполнено экспериментальное исследование алгоритмов, а именно:
  - выполнено сравнение производительности с Metanome для алгоритмов Spider и Sawfish;
  - выполнено исследование производительности для алгоритма Mind.

Код реализации открыт и доступен в репозитории[8].

[8] https://github.com/Desbordante/desbordante-core (дата обращения: 30 мая 2025 г.).

Список литературы

[1] Abedjan Ziawasch, Golab Lukasz, Naumann Felix. Profiling relational data: a survey // The VLDB Journal. — 2015. — aug. — Vol. 24, no. 4. — P. 557–581. — URL: http://dx.doi.org/10.1007/s00778-015-0389-y.

[2] Chomicki Jan, Marcinkowski Jerzy. Minimal-change integrity maintenance using tuple deletions // Information and Computation. — 2005. — Vol. 197, no. 1. — P. 90–121. — URL: https://www.sciencedirect.com/science/article/pii/S0890540105000179.

[3] Codd E. F. Further Normalization of the Data Base Relational Model // Research Report / RJ / IBM / San Jose, California. — 1971. — Vol. RJ909. — URL: https://api.semanticscholar.org/CorpusID:45071523.

[4] Cure Olivier. Improving the Data Quality of Drug Databases using Conditional Dependencies and Ontologies // J. Data and Information Quality. — 2012. — 10. — Vol. 4, no. 1. — 21 p. — URL: https://doi.org/10.1145/2378016.2378019.

[5] Data profiling with metanome / Thorsten Papenbrock, Tanja Bergmann, Moritz Finke et al. // Proceedings of the VLDB Endowment. — 2015. — aug. — Vol. 8, no. 12. — P. 1860–1863. — URL: http://dx.doi.org/10.14778/2824032.2824086.

[6] Desbordante: a Framework for Exploring Limits of Dependency Discovery Algorithms / Maxim Strutovskiy, Nikita Bobrov, Kirill Smirnov, George Chernishev // 2021 29th Conference of Open Innovations Association (FRUCT). — IEEE, 2021. — may. — URL: http://dx.doi.org/10.23919/FRUCT52173.2021.9435469.

[7] Fagin Ronald. Horn clauses and database dependencies (Extended Abstract) // Symposium on the Theory of Computing. — 1980. — URL: https://api.semanticscholar.org/CorpusID:6285434.

[8] Fagin Ronald. A Normal Form for Relational Databases That is Based on Domains and Keys // ACM Trans. Database Syst. — 1981. — sep. — Vol. 6, no. 3. — P. 387–415. — URL: https://doi.org/10.1145/319587.319592.

[9] Functional Dependency Generation and Applications in Pay-As-You-Go Data Integration Systems. / Daisy Zhe Wang, Xin Luna Dong, Anish Das Sarma et al. // WebDB. — 2009.

[10] Inclusion Dependency Discovery: An Experimental Evaluation of Thirteen Algorithms / Falco Dürsch, Axel Stebner, Fabian Windheuser et al. // Proceedings of the 28th ACM International Conference on Information and Knowledge Management. — CIKM '19. — New York, NY, USA : Association for Computing Machinery, 2019. — P. 219–228. — URL: https://doi.org/10.1145/3357384.3357916.

[11] Kaminsky Youri, Pena Eduardo, Naumann Felix. Discovering Similarity Inclusion Dependencies // Proceedings of the ACM on Management of Data. — 2023. — 05. — Vol. 1. — P. 1–24.

[12] Lopes Stéphane, Petit Jean-Marc, Toumani Farouk. Discovering Interesting Inclusion Dependencies: Application to Logical Database Tuning // Inf. Syst. — 2002. — 03. — Vol. 27. — P. 1–19.

[13] Marchi Fabien De, Lopes Stéphane, Petit Jean-Marc. Unary and n-ary inclusion dependency discovery in relational databases // Journal of Intelligent Information Systems. — 2009. — Vol. 32. — P. 53–73. — URL: https://api.semanticscholar.org/CorpusID:14322668.